

UNIVERSIDAD DEL VALLE DE GUATEMALA



Proyecto 1: Competencia de Modelación

Pedro Rubén Avila Cofiño, #23089

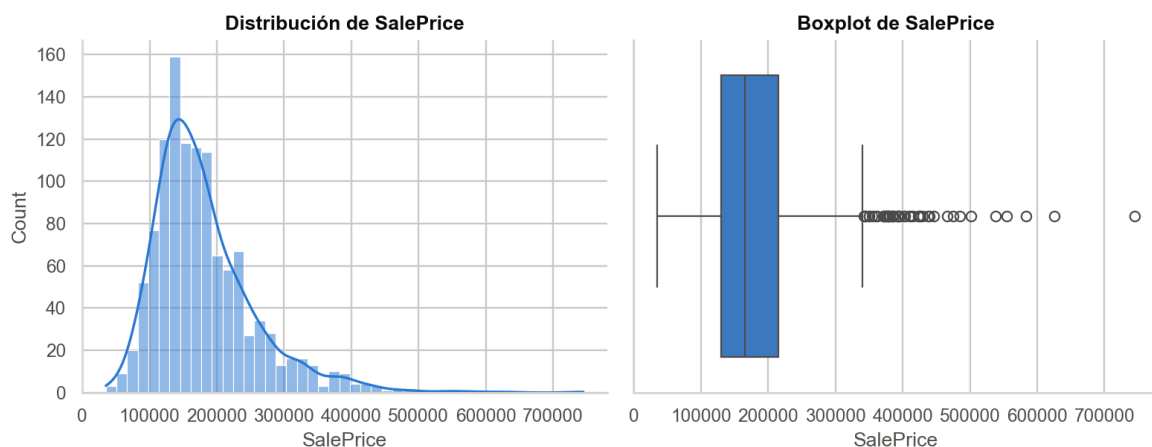
GUATEMALA 21 de agosto de 2026

1. Análisis exploratorio de datos (EDA)

El dataset utilizado corresponde a Ames Housing (House Prices), entregado por el curso como data/train.csv. Contiene 1,168 observaciones y 81 columnas: un identificador (Id), la variable objetivo (SalePrice, el precio de venta de la vivienda en USD) y 79 variables predictoras, de las cuales 36 son numéricas (continuas o discretas: áreas, año de construcción, número de baños, etc.) y 43 son categóricas u ordinales codificadas como texto (calidad de materiales, tipo de garage, vecindario, etc.). El curso también entregó por separado un archivo de 292 filas sin la columna SalePrice (data/test_features-1.csv), usado como banco de pruebas local para el pipeline de predicción antes del dataset held-out real del día de competencia.

1.1 Variable objetivo

SalePrice tiene media USD 181,442, mediana USD 165,000, desviación estándar USD 77,264 y rango [USD 34,900, USD 745,000]. Su distribución está sesgada a la derecha (skewness = 1.74): hay muchas más viviendas de precio medio/bajo que de precio muy alto, lo cual es típico en variables de precio.



Aplicar $\log(1 + \text{SalePrice})$ reduce el sesgo de 1.74 a 0.12 (prácticamente simétrico), lo cual es la transformación estándar recomendada para este dataset.

1.2 Valores nulos

19 de las 81 columnas tienen al menos un valor nulo. Las de mayor proporción son PoolQC, MiscFeature, Alley y Fence, con más del 80% de valores faltantes:

Variable	Nulos	% del total
PoolQC	1,162	99.5%
MiscFeature	1,122	96.1%
Alley	1,094	93.7%

Fence	935	80.1%
MasVnrType	683	58.5%
FireplaceQu	547	46.8%
LotFrontage	217	18.6%
GarageType	64	5.5%
GarageYrBlt	64	5.5%
GarageFinish	64	5.5%

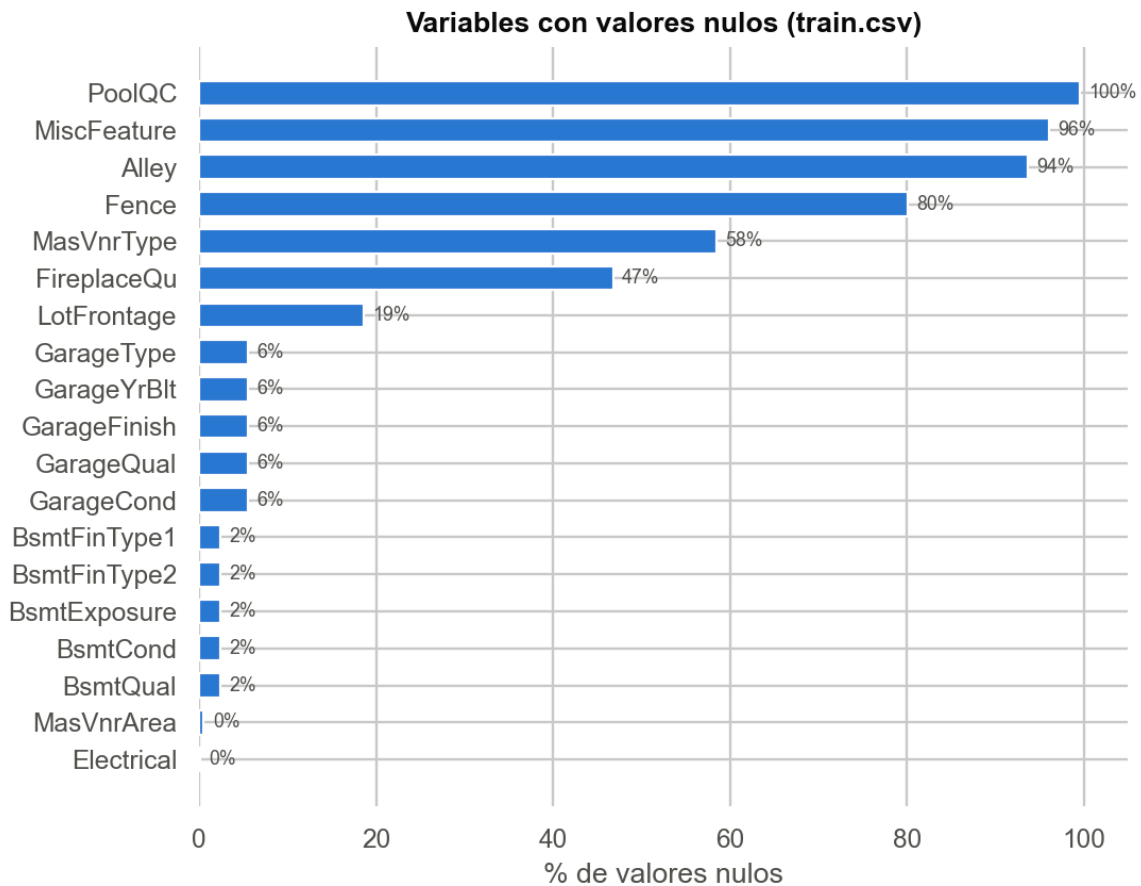


Figura 3. Porcentaje de valores nulos por variable (solo variables con al menos un nulo).

En el diccionario de datos de Ames Housing, la mayoría de estos nulos no son datos faltantes en el sentido estricto, sino que codifican la ausencia de la característica (p. ej. PoolQC = NaN significa "no tiene piscina", no "dato perdido"). El mismo patrón aplica a MiscFeature, Alley, Fence, FireplaceQu y a los grupos Garage* y Bsmt*, cuyos nulos coinciden exactamente entre sí (64 y 28 filas respectivamente), consistente con "no tiene garage" / "no tiene sótano". LotFrontage (18.6% nulo) es la única variable numérica con un porcentaje de nulos considerable y sí corresponde a un dato realmente no registrado.

Tratamiento aplicado: el pipeline de preprocesamiento (scripts/train.py, build_preprocessor) imputa las columnas numéricas con la mediana y las categóricas con la moda (most_frequent), dentro de un ColumnTransformer ajustado solo sobre el conjunto de entrenamiento para evitar fuga de información. Esta es una decisión pragmática y simple; una alternativa más fiel al significado real de estos nulos sería imputar una categoría explícita "None"/"Sin garage" en vez de la moda, lo cual se discute como mejora futura en la sección de conclusiones.

1.3 Valores atípicos (outliers)

Usando el criterio de rango intercuartílico (IQR, $1.5 \times \text{IQR}$) sobre las variables numéricas, las que presentan más observaciones atípicas son:

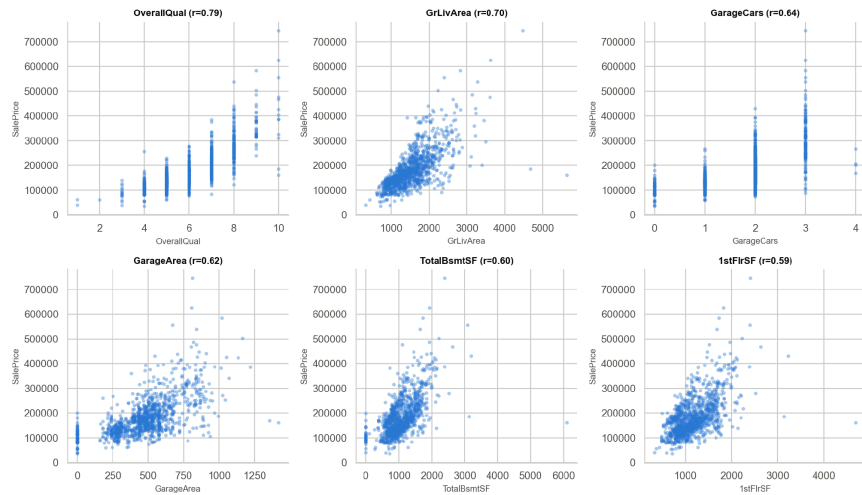
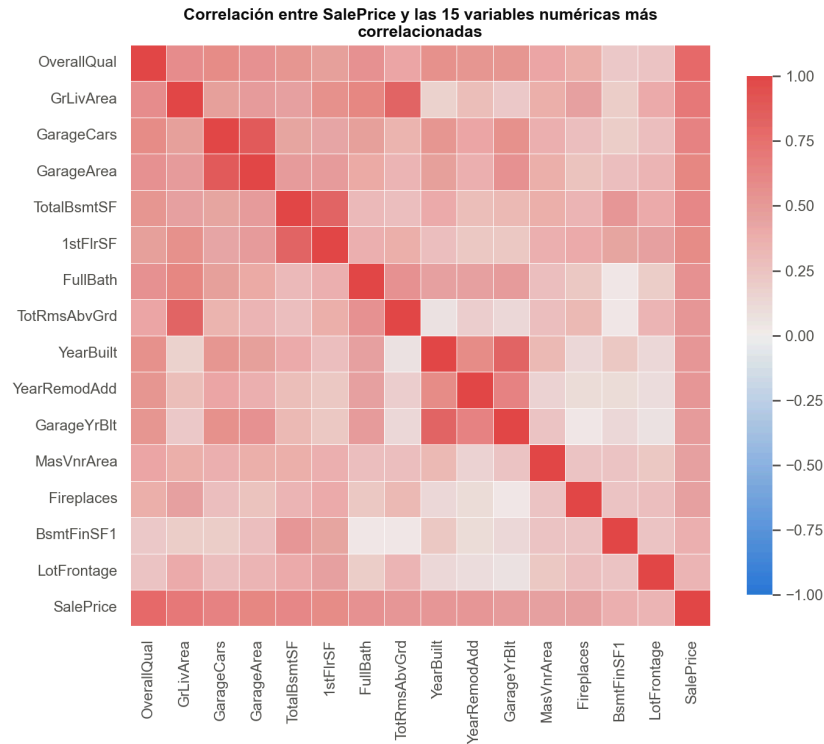
Variable	N° de outliers (IQR)
EnclosedPorch	160
BsmtFinSF2	131
OverallCond	99
ScreenPorch	97
MSSubClass	84
MasVnrArea	77
LotFrontage	68
BsmtHalfBath	66

El caso más relevante y mejor documentado en este dataset es GrLivArea (área habitable sobre nivel de suelo): existen 3 viviendas con GrLivArea > 4,000 pies², todas con OverallQual = 10 (la calidad máxima). Dos de ellas (GrLivArea de 4,676 y 5,642 pies²) se vendieron por solo USD 184,750 y USD 160,000 respectivamente, muy por debajo de lo esperado para su tamaño y calidad, mientras que la tercera (GrLivArea = 4,476) se vendió por USD 745,000, el precio máximo de todo el dataset. Estas dos primeras observaciones son un caso conocido de la literatura sobre este dataset (posibles ventas atípicas, p. ej. entre familiares o de propiedades incompletas) y suelen recomendarse para exclusión; en este proyecto se optó por no eliminarlas manualmente, ya que el pipeline no incluye un paso explícito de remoción de outliers, lo cual se retoma en la discusión de limitaciones.

1.4 Correlaciones con la variable objetivo

Las variables numéricas más correlacionadas (linealmente) con SalePrice son:

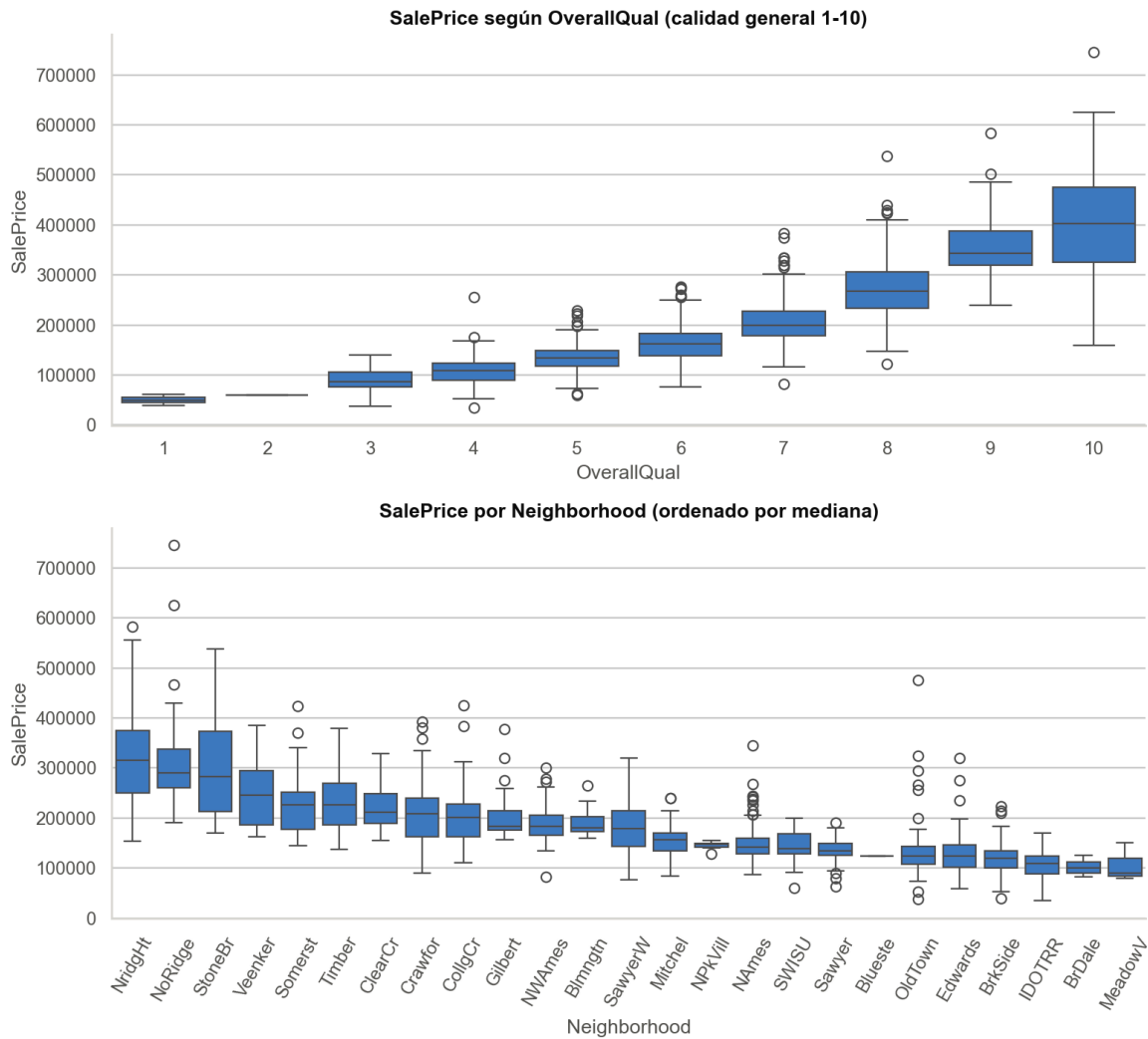
Variable	Correlación con SalePrice
OverallQual	0.786
GrLivArea	0.696
GarageCars	0.641
GarageArea	0.624
TotalBsmtSF	0.598
1stFlrSF	0.588
FullBath	0.553
TotRmsAbvGrd	0.520
YearBuilt	0.517
YearRemodAdd	0.509



OverallQual (calidad general de materiales y acabados, escala 1-10) es, por un margen amplio, el mejor predictor individual ($r = 0.79$), seguido de variables de tamaño (GrLivArea, GarageCars/GarageArea, TotalBsmtSF, 1stFlrSF). También se observan grupos de variables fuertemente correlacionadas entre si, es decir, cierta redundancia/multicolinealidad entre features, algo que un MLP maneja razonablemente bien pero que sí sería relevante para un modelo lineal.

1.5 Variables categóricas

Entre las variables categóricas, OverallQual y Neighborhood muestran la relación más clara y monótona con el precio: a mayor calidad general, mayor precio (con dispersión creciente en las categorías altas), y los vecindarios se ordenan en una jerarquía de precio bastante estable (de NridgHt/NoRidge/StoneBr en la parte alta a MeadowV/BrDale/IDOTRR en la parte baja).



1.6 Decisiones de preprocesamiento derivadas del EDA

- Imputación: mediana para numéricas, moda para categóricas (SimpleImputer dentro de un ColumnTransformer, ajustado solo con datos de entrenamiento).
- Estandarización (media 0, varianza 1) de todas las variables numéricas con StandardScaler, necesaria porque las escalas originales son muy dispares (áreas en cientos/miles de pies², años en ~1900-2010, conteos pequeños de baños/habitaciones) y un MLP entrenado con gradientes es sensible a estas diferencias de escala.

- Codificación one-hot (OneHotEncoder, handle_unknown="ignore") para las categóricas, sin asumir ningún orden entre categorías nominales; esto evita introducir una relación de orden falsa (p. ej. entre nombres de vecindario) a costa de aumentar la dimensionalidad de entrada.
- Estandarización del target (SalePrice) con un StandardScaler independiente, revertida antes de reportar RMSE; ver sección de metodología para la justificación de esta decisión, que fue en realidad producto de una iteración fallida documentada en la sección 3.
- No se eliminaron outliers ni se aplicó transformación logarítmica al target en la versión final del pipeline; ambas son mejoras identificadas pero no implementadas (sección 5).

2. Metodología de desarrollo

2.1 Arquitectura

El modelo es un Multi-Layer Perceptron implementado en PyTorch (scripts/train.py, clase MLP): una pila de capas totalmente conectadas (nn.Linear), cada una seguida de una función de activación configurable (ReLU, Tanh o GELU) y una capa de Dropout, terminando en una capa lineal de salida de una sola neurona (regresión de SalePrice). El número de capas ocultas, su ancho, la activación y el dropout son parámetros del archivo de configuración scripts/config.toml, lo que permitió iterar sobre la arquitectura sin modificar código. Se probaron dos arquitecturas principales: una más pequeña ([64, 32] neuronas, ReLU) y una más profunda y ancha ([256, 128, 64] neuronas, GELU), descritas en la sección 3.

2.2 División de datos

Para poder documentar el historial de iteraciones de este informe con una métrica de generalización adicional a la de validación (y poder hacer un análisis de errores con datos que el modelo no vio en ningún momento del entrenamiento ni de la selección de hiperparámetros), se realizó un split adicional de análisis 70/15/15 (train=817, val=175, test=176) usado únicamente para comparar iteraciones en la sección 3; el modelo final entregado para la competencia se re-entrena con el split 80/20 oficial del pipeline de producción sobre el 100% de data/train.csv disponible.

2.3 Función de pérdida, optimizador e hiperparámetros

- Función de pérdida: error cuadrático medio (nn.MSELoss), calculado en el espacio estandarizado del target; el RMSE que se reporta y compara siempre se calcula revertido a unidades de USD.
- Optimizador: Adam, con tasa de aprendizaje y weight decay (L2) como hiperparámetros de scripts/config.toml.
- Tamaño de batch: hiperparámetro configurable; se probaron 8 y 32.
- Épocas: hasta 200-300, con early stopping (ver regularización).

2.4 Regularización

- Dropout dentro de cada bloque oculto (0.1 en la arquitectura pequeña, 0.2 en la grande).
- Weight decay (L2) en el optimizador Adam.
- Early stopping: se guarda el mejor estado del modelo según RMSE de validación (en unidades reales de USD, no en el espacio estandarizado) y el entrenamiento se detiene tras 20 épocas sin mejora (early_stopping_patience = 20), evitando así elegir un modelo sobreajustado de una época tardía.

No se usó batch normalization ni validación cruzada en este proyecto; ambas quedan identificadas como trabajo futuro en la sección 5.

2.5 Un hallazgo de metodología: la estandarización del target

Durante el desarrollo se detectó que, al pasar de la arquitectura pequeña ([64, 32], sin estandarizar el target) a la arquitectura grande, se introdujo estandarización del target (StandardScaler sobre SalePrice) pero se mantuvieron valores de tasa de aprendizaje muy pequeños (5e-5) heredados de pruebas anteriores, junto con un batch_size de 8 y un weight_decay de 1e-6. El resultado fue un RMSE de validación notablemente peor de lo esperado dado el tamaño de la red. La causa: al estandarizar el target, los gradientes quedan en una escala "normal" (~unitaria) en vez de la escala de decenas de miles de dólares de SalePrice; usar una tasa de aprendizaje pensada para gradientes en escala de dólares resulta demasiado pequeña una vez el target está en escala estándar, y el modelo prácticamente no se aleja de su inicialización aleatoria en el número de épocas disponible. La corrección (learning_rate = 1e-3, batch_size = 32, weight_decay = 1e-4) y su efecto medido se documentan como iteraciones explícitas en la sección 3.

3. Resultados de iteraciones

Se documentan a continuación 5 iteraciones representativas del proceso de búsqueda, cada una cambiando una o dos variables respecto a la anterior para poder atribuir el efecto de cada cambio. Todas se entrenaron y evaluaron sobre el split de análisis 70/15/15 descrito en la sección 2.2, con semilla fija, para que fueran directamente comparables entre sí.

Iter.	Cambio principal	Arquitectura	Épocas (usadas/máx)	Parám.	RMSE train	RMSE val	RMSE test	Tiempo (s)
I1	Punto de partida: red pequeña, target sin escalar	[64,32] relu	200/200	20,097	31,504	32,384	24,648	2.7
I2	+ estandarizar SalePrice (y scaler)	[64,32] relu	34/200	20,097	15,940	27,363	21,080	0.4
I3	+ red [256,128,64] GELU,	[256,128,64] gelu	49/200	113,153	8,540	27,440	20,368	1.2

	dropout 0.2							
I4	batch 8, lr 5e-5, weight_de cay 1e-6	[256,128,6 4] gelu	71/300	113,153	13,336	27,148	21,804	5.6
I5	fix: batch 32, lr 1e-3, weight_de cay 1e-4	[256,128,6 4] gelu	49/300	113,153	8,923	27,151	20,025	1.3

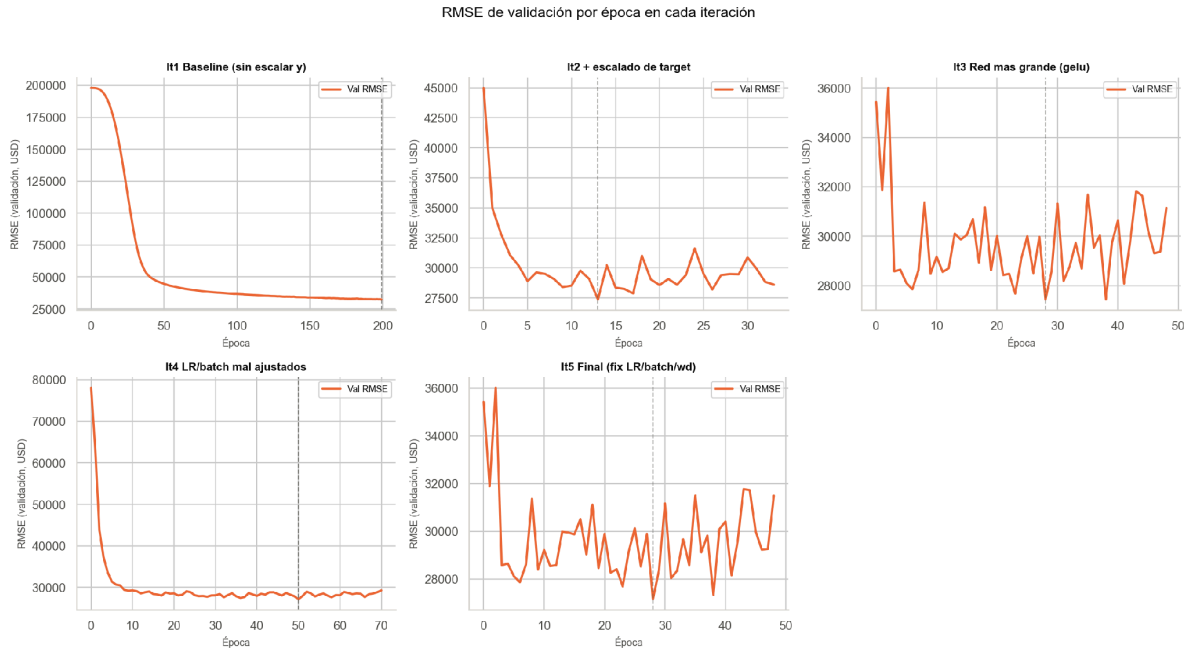
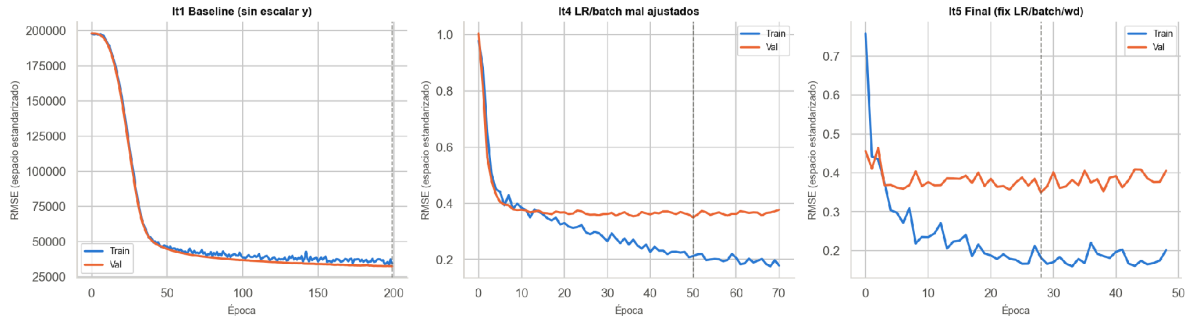


Figura 7. RMSE de validación (USD) por época en cada una de las 5 iteraciones. La línea vertical marca la época del mejor modelo (usada para early stopping).



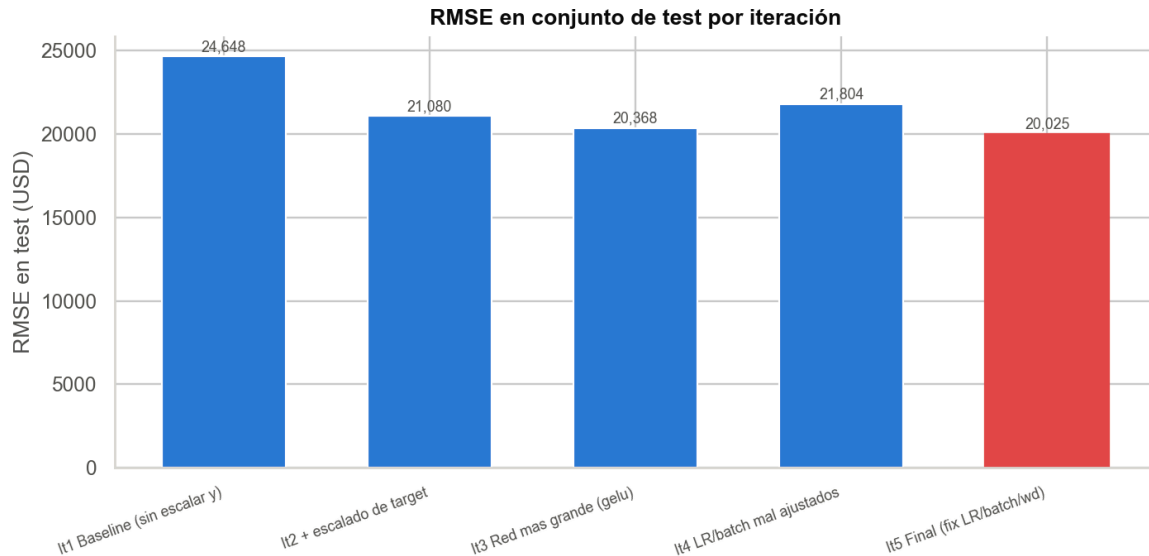


Figura 9. RMSE en el conjunto de prueba (test, 15% held-out del split de análisis) para cada iteración.

3.1 Problemas encontrados y cómo se abordaron

- Convergencia lenta / posible underfitting en I1: con el target sin estandarizar, el RMSE de validación seguía bajando lentamente al llegar al límite de 200 épocas sin activar el early stopping (mejor época = 199), señal de que el modelo aún tenía margen para seguir mejorando con más tiempo de entrenamiento o una tasa de aprendizaje mayor.
- Overfitting en la red grande (I3 e I5): la brecha entre RMSE de entrenamiento (~USD 8,540-8,923) y de validación (~USD 27,440) es considerable: el modelo memoriza patrones del set de entrenamiento que no generalizan igual de bien. Dropout (0.2) y weight decay ($1e-4$) atenúan parcialmente esta brecha pero no la eliminan, algo esperable dado que el dataset de entrenamiento (817 filas en el split de análisis) es pequeño en relación con los 113,153 parámetros de la red.
- Hiperparámetros mal calibrados (I4): la combinación de `batch_size = 8`, `learning_rate = 5e-5` y `weight_decay = 1e-6` no colapsó el entrenamiento, pero sí lo hizo notablemente más lento en tiempo de reloj (5.6s frente a 1.2s de I3, por el mayor número de mini-batches por época) y ligeramente peor en test (USD 21,804 frente a USD 20,368 de I3). Se abordó revirtiendo los tres valores a los usados en I3/I5 (`batch 32`, `lr 1e-3`) y añadiendo un `weight_decay` más significativo ($1e-4$) para compensar el mayor tamaño del modelo.

4. Discusión de resultados

4.1 Comparación entre iteraciones

El cambio con mayor impacto fue estandarizar el target (I1 $\rightarrow$ I2): con la misma arquitectura pequeña, el RMSE de test bajó de USD 24,648 a USD 21,080 (-14%) y la convergencia pasó de necesitar el presupuesto completo de 200 épocas a estabilizarse en apenas $\sim$ 13-30 épocas. Aumentar la capacidad del modelo (I2 $\rightarrow$ I3, red [256,128,64] con GELU) aportó una mejora adicional pero más modesta en test (USD 21,080 $\rightarrow$ USD 20,368), a costa de un RMSE de entrenamiento mucho menor (USD 8,540) que evidencia mayor sobreajuste.

4.2 Análisis de errores del modelo final

Evaluando el modelo de la iteración final (I5) sobre las 176 viviendas del conjunto de prueba del split de análisis: el residuo promedio es de USD -2,031 (prácticamente centrado en cero, sin sesgo sistemático de sobre- o subestimación) con una desviación estándar de USD 19,921, y un error porcentual absoluto medio (MAPE) de 8.3%.

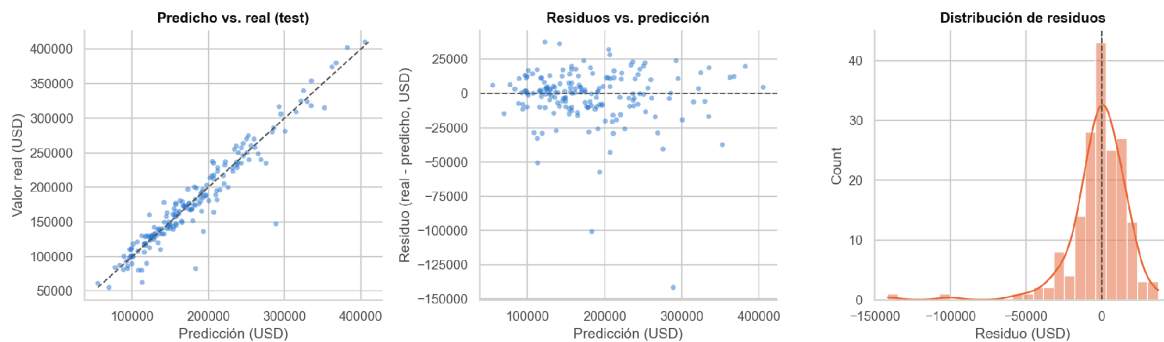


Figura 10. Predicho vs. real, residuos vs. predicción, y distribución de residuos del modelo final (I5) sobre el conjunto de prueba.

El caso de mayor error absoluto corresponde a una vivienda vendida en USD 147,000 para la que el modelo predijo USD 288,766 (sobreestimación de USD 141,766); en el gráfico de residuos vs. predicción (Figura 10, centro) se observan dos o tres casos claramente separados del resto, lo cual sugiere viviendas con combinaciones de características atípicas para su rango de precio (posiblemente relacionadas con los outliers de GrLivArea identificados en el EDA) más que un problema sistemático del modelo.

4.3 Limitaciones del enfoque y del dataset

- El dataset de entrenamiento disponible (1,168 filas) es pequeño para una red de 113,153 parámetros; esto se refleja en la brecha train/validación observada desde I3 en adelante y en la variabilidad considerable del RMSE de validación entre épocas cercanas.

4.4 Complejidad del modelo vs. capacidad de generalización

Los resultados muestran el trade-off clásico complejidad-generalización de forma bastante directa: la red pequeña (20,097 parámetros) generaliza con una brecha train/val pequeña pero un techo de desempeño más alto (RMSE de validación ~27,000-32,000 según si el target está o no estandarizado); la red grande (113,153 parámetros) baja el techo de error de validación/test pero solo alcanzándolo a costa de un sobreajuste claro (RMSE de entrenamiento 3-4 veces menor que el de validación). El uso de dropout, weight decay y, sobre todo, early stopping fueron necesarios para que la red grande generalizara mejor que la pequeña en vez de simplemente memorizar más rápido; sin early stopping, es probable que el desempeño de validación de la red grande se hubiera deteriorado bastante antes de las 300 épocas de presupuesto.

5. Conclusiones

Un RMSE de aproximadamente USD 26,700-27,000 representa cerca del 15% del precio promedio de una vivienda en este dataset (USD 181,442) y un error porcentual medio (MAPE) del 8.3% sobre el conjunto de prueba del análisis de iteraciones: un desempeño razonable para un MLP simple sin ingeniería de features avanzada, aunque con margen de mejora claro frente a las técnicas más sofisticadas típicas de este dataset (modelos de árboles con boosting, ensambles, transformación logarítmica del target).

5.1 Principales aprendizajes

- La estandarización del target no es opcional cuando se usa MSE con un optimizador y tasa de aprendizaje pensados para gradientes de magnitud unitaria: fue, con diferencia, el cambio individual de mayor impacto de todo el proyecto, muy por delante de cualquier cambio de arquitectura.
- Cambiar varios hiperparámetros de tasa de aprendizaje/batch/regularización a la vez (como ocurrió al pasar a la red grande) dificulta saber cuál de ellos es responsable de una mejora o un empeoramiento; aislar los cambios de uno en uno (como se hizo retroactivamente en la sección 3) fue necesario para entender qué había pasado realmente.

5.2 Trabajo futuro

- Transformar el target con $\log(1 + \text{SalePrice})$ en vez de solo estandarizarlo, dado que reduce el sesgo de 1.74 a 0.12 y podría corregir el patrón de MAPE decreciente con el precio detectado en la sección 4.2.
- Imputar los nulos de variables tipo PoolQC/Alley/Fence/Garage*/Bsmt* con una categoría explícita de "ausente" en vez de la moda, para no distorsionar su relación real con el precio.
- Evaluar la remoción o el downweighting de los 2-3 outliers de GrLivArea > 4,000 identificados en el EDA.
- Añadir validación cruzada (k-fold) para que las comparaciones entre iteraciones sean menos sensibles a una sola partición train/val, y repetir las iteraciones clave con varias semillas.

- Explorar batch normalization y búsqueda más sistemática de hiperparámetros (grid/random search u optimización bayesiana) en vez de la búsqueda manual guiada por los comentarios de configuración usada en este proyecto.

6. Repositorio

https://github.com/PeDro0210AVI/Proj_01_MLP